Solution: Sudoku Solver

Problem Statement

Write a program to solve a Sudoku puzzle by filling the empty cells.

A sudoku solution must satisfy all of the following rules:

- Each of the digits 1-9 must occur exactly once in each row.
- Each of the digits 1-9 must occur exactly once in each column.
- Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid.

The '.' character indicates empty cells.

Example 1:

Input:

```
{'5', '3', '.', '.', '7', '.', '.', '.', '.'},
{'6', '.', '.', '1', '9', '5', '.', '.', '.'},
{'.', '9', '8', '.', '.', '.', '.', '6', '.'},
{'8', '.', '.', '.', '6', '.', '.', '.', '3'},
{'4', '.', '.', '8', '.', '3', '.', '.', '1'},
{'7', '.', '.', '.', '2', '.', '.', '.', '6'},
{'.', '6', '.', '.', '.', '.', '2', '8', '.'},
{'.', '.', '.', '4', '1', '9', '.', '.', '5'},
{'.', '.', '.', '.', '8', '.', '.', '7', '9'}
```

Output:

```
{'5'', '3'', '4'', '6'', '7'', '8'', '9'', '1'', '2''},
{'6'', '7'', '2'', '1'', '9'', '5'', '3'', '4'', '8''},
{'1'', '9'', '8'', '3'', '4'', '2'', '5'', '6'', '7''},
{'8'', '5'', '9'', '7'', '6'', '1'', '4'', '2'', '3''},
{'4'', '2'', '6'', '8'', '5'', '3'', '7'', '9'', '1''},
{'7'', '1'', '3'', '9'', '2'', '4'', '8'', '5'', '6''},
{'9'', '6'', '1'', '5'', '3'', '7'', '2'', '8'', '4''},
{'2'', '8'', '7'', '4'', '1'', '9'', '6'', '3'', '5''},
{'3'', '4'', '5'', '2'', '8'', '6'', '1'', '7'', '9''}
```

Explanation: The given output is the only valid Sudoku solution.

Solution

Sudoku is a popular puzzle game that involves filling in a 9x9 grid with numbers such that each column, each row, and each of the nine 3x3 sub-grids that compose the grid contains all of the digits from 1 to 9. One way to solve a sudoku puzzle is by using backtracking, which is a form of brute-force search.

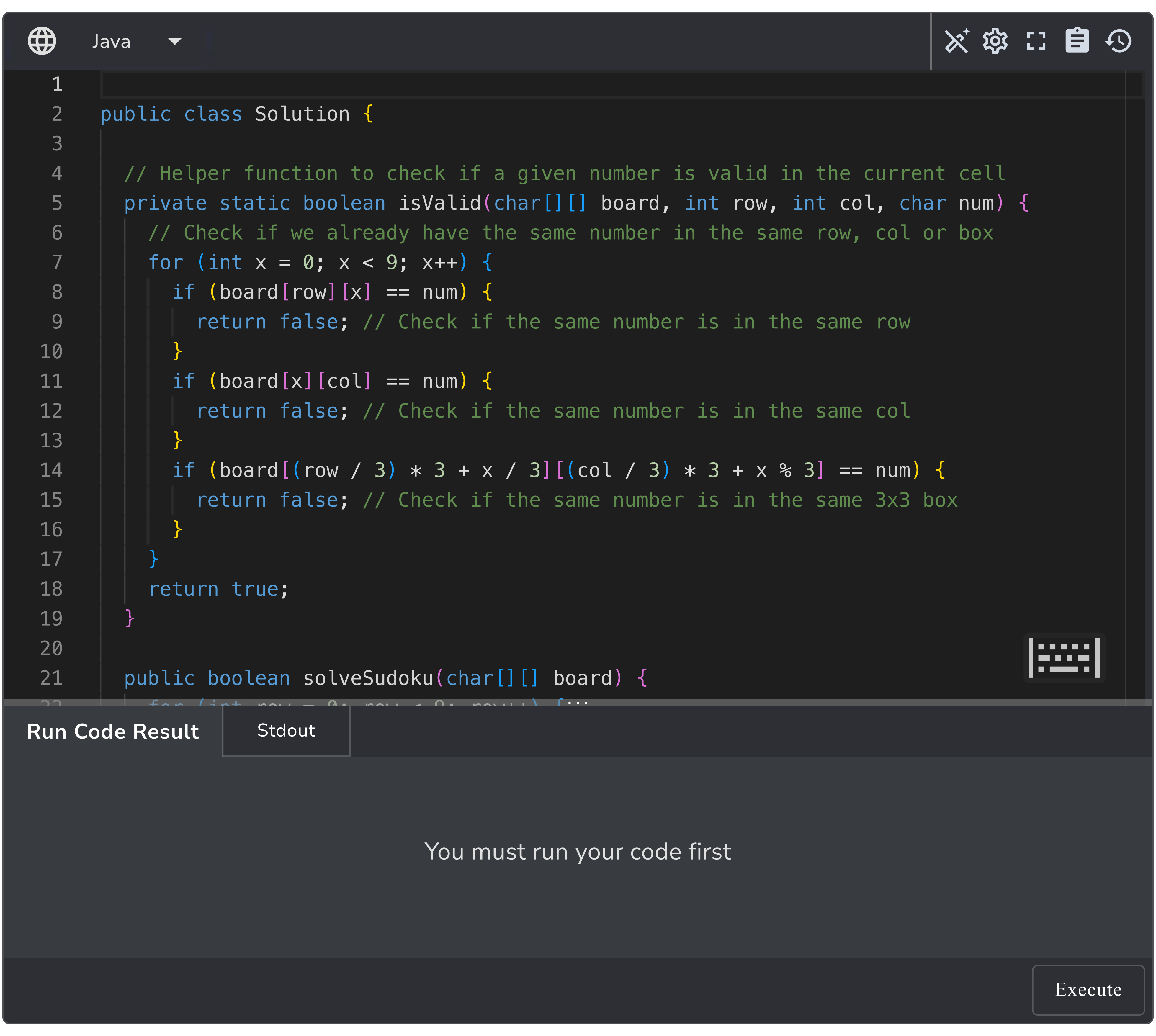
Here are the steps to solve a sudoku puzzle using backtracking:

1. Start with an empty 9x9 grid and fill in the numbers that are given in the puzzle.
2. From left to right and top to bottom, find the first empty cell (denoted by a `.`) in the grid.
3. Try out every number from 1 to 9 as a possible solution for the empty cell.
4. Before filling in a number, check if it is a valid solution by checking if the number is already present in the same row, column, or 3x3 sub-grid.
5. If the number is valid, fill it in the cell and recursively call the function to move to the next empty cell.
6. If the number is not valid, backtrack to the previous state and try a different number.
7. If a valid solution is found, return true. If no solution is found, return false.
8. If the function returns true, the grid is filled with a valid solution. Else, the puzzle is unsolvable.

Code

The function `solveSudoku` starts by iterating through the board and checking for empty cells represented by '.' character. When an empty cell is found, the function tries every number from 1-9 by using a nested loop. Before filling the cell with a number, the function checks if the number is valid in the current cell by using the helper function `isValid`. The function `isValid` checks if the same number is already present in the same row, column or 3x3 box. If the number is valid, the cell is filled with the number and the function recursively calls itself to solve the rest of the board.

If the current number doesn't lead to a solution, the function backtracks by emptying the cell and trying the next number. If all the numbers have been tried and none of them lead to a solution, the function returns false. If the board is completely filled, the function returns true, indicating that a solution has been found.



```java

public class Solution {

  // Helper function to check if a given number is valid in the current cell
  private static boolean isValid(char[][] board, int row, int col, char num) {
    // Check if we already have the same number in the same row, col or box
    for (int x = 0; x < 9; x++) {
      if (board[row][x] == num) {
        return false; // Check if the same number is in the same row
      }
      if (board[x][col] == num) {
        return false; // Check if the same number is in the same col
      }
      if (board[(row / 3) * 3 + x / 3][(col / 3) * 3 + x % 3] == num) {
        return false; // Check if the same number is in the same 3x3 box
      }
    }
    return true;
  }

  public boolean solveSudoku(char[][] board) {
```

You must run your code first

Time Complexity

since the board size is fixed, the time complexity is $O(1)$, as there is no variable input.

Though let's discuss the number of operations needed: $(9!)^9$

Let's consider one row where we have 9 cells to fill. There are not more than 9 possibilities for the first number to put, not more than 9×8 for the second one, not more than 9×8×7 for the third one, and so on. In total, that results in not more than $(9!)$ possibilities for just one row; this means not more than $(9!)^9$ operations in total.

If we compare the brutefore and backtracking algorithms:

- 9^{81} = 1966270504755529136180759085269121162831034509442147669273154155379663911968099 for the brute force,
- and $(9!)^9$=109110688415571316480344899355894085582848000000000 for the standard backtracking, i.e. the number of operations is reduced in 10^{27} times!

Space Complexity

The board size is fixed, and the space is used to store the board containing 81 cells, hence the time complexity is $O(1)$.

← Previous
Sudoku Solver (Hard)
Next →
Factor Combinations (Medium)
☐ Mark as Completed